

Grupo:

ALLAN JACKSON SILVA OLIVEIRA
CALLEB SOARES DA COSTA
GABRIEL DA SILVA VIEIRA
GUSTAVO LUIS LEITE
GUSTAVO PROVENZANO DE SOUSA

Projeto Integrador:

Brasil

2026

Grupo:

ALLAN JACKSON SILVA OLIVEIRA
CALLEG SOARES DA COSTA
GABRIEL DA SILVA VIEIRA
GUSTAVO LUIS LEITE
GUSTAVO PROVENZANO DE SOUSA

Projeto Integrador:

Projeto Integrador 1

Centro Universitário Senac
Análise e Desenvolvimento de Sistemas

Orientador: Orlando C. Patriarcha

Brasil
2026

Resumo

Este trabalho descreve o desenvolvimento de uma aplicação em linguagem Java que funciona como um motor (engine) um (*quiz*) pedagógico. O projeto foi desenvolvido como parte integrante do primeiro semestre do curso de Análise e Desenvolvimento de Sistemas, focando na aplicação prática de fundamentos de algoritmos e estruturação de código.

A implementação destaca-se pela utilização de métodos estáticos para a modularização do sistema, permitindo a separação clara entre a interface de usuário (CLI - *Command Line Interface*), as regras de negócio e a gestão de fluxo de telas. Foram aplicados conceitos fundamentais como estruturas de repetição (**do-while**, **for**) para controle de interface, estruturas condicionais (**switch-case**, **if-else**) para o processamento de escolhas e tratamento de dados via classe **Scanner**. O software conta com um sistema de validação de respostas que garante a integridade da experiência do usuário, impedindo entradas de dados fora dos parâmetros esperados.

Os resultados obtidos evidenciam a eficácia do uso de métodos para a criação de códigos reutilizáveis e de fácil manutenção, além de demonstrar a viabilidade de criar experiências interativas complexas dentro das limitações do terminal, consolidando os pilares necessários para o avanço em Programação Orientada a Objetos (POO).

Palavras-chave: Java; Algoritmos; Modularização; RPG; Desenvolvimento de Software.

Lista de ilustrações

Figura 1 – Fluxograma Navegação do Sistema	7
Figura 2 – Exibição da introdução histórica e regras do quiz.	9
Figura 3 – Exibição das regras do quiz.	9
Figura 4 – Interface do Placar de Líderes	9
Figura 5 – Tela de Introdução da Partida	11
Figura 6 – Tela de um desafio do quiz	13
Figura 7 – Entrada de dados incorretos	21

Sumário

1	INTRODUÇÃO	5
1.1	Objetivo	5
1.2	Finalidade	5
2	DESENVOLVIMENTO DO SISTEMA	7
2.1	Fluxo e Telas do Sistema	7
2.1.1	Tela de Título (Menu Principal)	8
2.1.2	Tela de Regras	9
2.1.3	Placar de Líderes (Top 10 Scores)	9
2.1.4	Função Iniciar Jogo	11
2.2	Arquitetura e Modularização	15
2.2.1	Dados da Aplicação	15
2.2.2	Separação de camada de renderização	17
2.2.3	Validação da entrada do usuário	19
2.2.4	Algoritmo de Fisher-Yates	22
2.2.5	Função adicionarDoisPontos()	23
2.2.6	Função limparTela()	24
2.2.7	Função pausar()	24
2.2.8	Função contemValor()	25
2.3	Código Fonte	27
3	CONCLUSÃO	29
	REFERÊNCIAS	31

1 Introdução

O avanço das metodologias de ensino de programação tem demonstrado que a aplicação prática de conceitos teóricos é fundamental para a formação de desenvolvedores de software. No contexto do curso de Análise e Desenvolvimento de Sistemas, o desafio de transpor algoritmos abstratos para aplicações funcionais é uma etapa crítica do aprendizado. Este projeto apresenta o desenvolvimento de um jogo no estilo *Quiz*, utilizando a linguagem Java, denominado *Chute Certo*. A aplicação foi concebida com o intuito pedagógico de auxiliar no processo de aprendizagem de jovens estudantes do Ensino Fundamental 1

1.1 Objetivo

O projeto tem como metas principais:

- **Geral:** Desenvolver um software educacional interativo baseado em interface de terminal, aplicando os fundamentos da linguagem Java para a criação de um jogo de perguntas e respostas voltado ao Ensino Fundamental 1.
- **Específicos:**
 - Implementar uma estrutura modularizada através de métodos estáticos para organizar o fluxo do sistema.
 - Aplicar estruturas de controle (laços de repetição e condicionais) para gerenciar a dinâmica do jogo.
 - Garantir a integridade da aplicação através de mecanismos de validação de dados inseridos pelo usuário.
 - Consolidar o uso de ferramentas básicas da biblioteca padrão do Java, como a classe `Scanner`.

1.2 Finalidade

A finalidade deste trabalho é puramente acadêmica e pedagógica, servindo como instrumento de avaliação e consolidação dos conhecimentos adquiridos no primeiro semestre do curso. Busca-se demonstrar a viabilidade técnica de criar uma ferramenta de entretenimento educativa utilizando recursos básicos de computação, provando que a robustez do Java permite a criação de sistemas organizados e funcionais mesmo em ambientes CLI (*Command Line Interface*).

2 Desenvolvimento do Sistema

2.1 Fluxo e Telas do Sistema

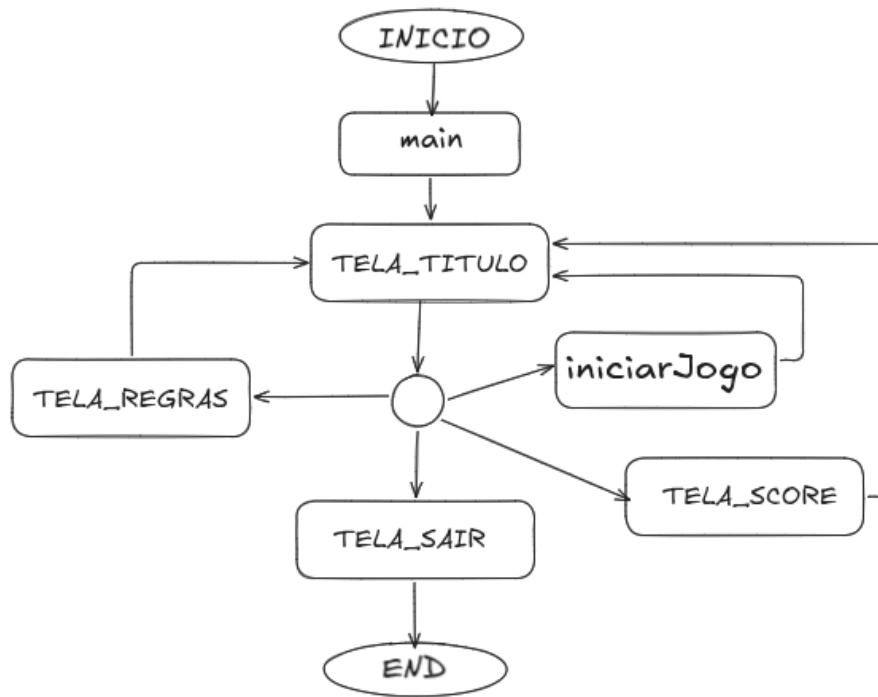


Figura 1 – Fluxograma Navegação do Sistema

No sistema *Chute Certo*, toda a interação entre o usuário e o software ocorre de modo textual via terminal. O fluxo de navegação inicia-se na Tela de Título, mapeada pela constante `TELA_TITULO`, onde o usuário seleciona uma opção numérica correspondente à funcionalidade desejada:

- (1) **Iniciar Jogo:** Redireciona o usuário para o *quiz*, onde responderá às questões;
- (2) **Regras:** Apresenta as instruções detalhadas de jogabilidade;
- (3) **Placar de Líderes (*Leaderboard*):** Exibe as 10 maiores pontuações registradas;
- (4) **Sair:** Finaliza a execução da aplicação.

```
1 public static void telaTitulo() {
2     boolean limparTela = false;
3     while(true) {
4         desenharTela(TELA_TITULO, limparTela);
5         limparTela = true;
6
7         int op = lerOpcoes("Escolha", new int[]{1, 2, 3, 4});
8
9         switch (op) {
10             case 1 -> iniciarJogo();
11             case 2 -> telaRegras();
12             case 3 -> telaScore();
13             case 4 -> sair();
14         }
15     }
16 }
```

Listing 2.1 – Código tela título.

O fluxo foi projetado de forma cíclica: todas as opções, com exceção da escolha de encerramento, retornam o usuário ao menu principal. Essa lógica é gerenciada pelo método estático `telaTitulo()`, invocado na função principal (*main*) dentro de um laço de repetição (*while loop*) contínuo, que é interrompido estritamente pela chamada da função `sair()`. Dentro do método `sair()` é feita uma chamada de `System.exit()`, que encerra o programa.

2.1.1 Tela de Título (Menu Principal)

`TELA_TITULO` é o ponto de partida do software. Em razão das limitações visuais inerentes ao terminal, optou-se pela utilização da técnica de **ASCII Art** para a apresentação do título do jogo, visando capturar a atenção do público-alvo. Adicionalmente, a interface exibe um menu numérico intuitivo que disponibiliza as quatro opções de navegação do sistema, conforme ilustrado na Figura 2.

O sistema gerencia um painel com os dez melhores resultados obtidos. Essa classificação é exibida ao usuário antes do início do jogo e reapresentada ao término da partida. Caso o jogador alcance uma pontuação elegível para o ranking, o sistema permite a inserção de um identificador (nickname) de até três letras. Após a validação do nome do estudante por meio de uma expressão regular (Regex), a tabela de recordes é atualizada e exibida.(Figura 4).

```
1 public static void desenharScore() {
2     System.out.println("=====");
3     System.out.println("          TOP 10 SCORES          ");
4     System.out.println("=====");
5
6     if (fimDaPartida) {
7         System.out.println("Sua pontuação: " + pontuacao);
8         System.out.println("-----");
9     }
10
11     for (int i = 0; i < 10; i++) {
12         if (nomesTop10[i] != null) {
13             System.out.printf("%2d° Lugar: %-3s - %3d pts\n", (i
+ 1), nomesTop10[i].toUpperCase(), pontosTop10[i]);
14         } else {
15             System.out.printf("%2d° Lugar: ---\n", (i + 1));
16         }
17     }
18     System.out.println("=====\n");
19 }
```

Listing 2.2 – Código da Tela de Score

Para garantir a legibilidade, o painel é formatado utilizando `System.out.printf`, o que assegura o alinhamento vertical perfeito entre as colunas de classificação (1º ao 10º lugar), as iniciais do aluno e as respectivas pontuações.

2.1.4 Função Iniciar Jogo

Compreende os métodos `main()`, `telaTitulo()`, `iniciarJogo()` e `setup()`. São responsáveis por inicializar as variáveis, invocar o embaralhamento e ditar o encadeamento das fases do quiz.

```
1 public static void iniciarJogo() {  
2     setup();  
3     telaIntroducao();  
4     limparTela();  
5  
6     for(int i = 0; i < PERGUNTAS.length; i++) {  
7         telaDesafio(i);  
8     }  
9  
10    telaFinalizar();  
11 }
```

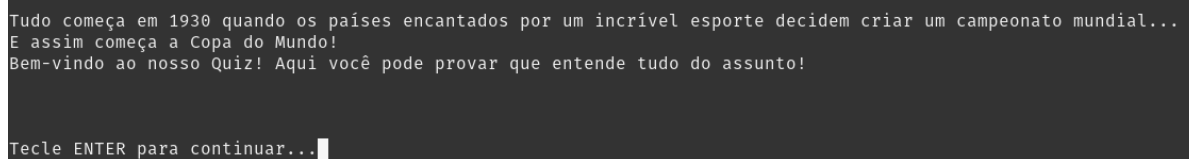
Listing 2.3 – Controle do ciclo de vida da partida.

A função `setup()` é responsável por estabelecer o estado inicial do sistema antes da execução da lógica principal da partida. Esse método realiza a configuração do ambiente e a reinicialização de variáveis de controle globais. Ela reinicia a variável de pontuação e faz o embaralhamento dos desafios - questões do quiz - para cada nova partida, sendo esta etapa indispensável para garantir o determinismo do programa, assegurando que o progresso de sessões anteriores não interfira em uma nova jogada.

```
1 public static void setup() {  
2     pontuacao = 0;  
3     embaralharQuestoes(PERGUNTAS, ALTERNATIVAS, RESPOSTAS);  
4 }
```

Listing 2.4 – Função Setup

Em seguida, é invocada a função `telaIntroducao()`, responsável por exibir um breve texto que contextualiza o início da partida e busca engajar o jogador. (Figura 5)



```
Tudo começa em 1930 quando os países encantados por um incrível esporte decidem criar um campeonato mundial...  
E assim começa a Copa do Mundo!  
Bem-vindo ao nosso Quiz! Aqui você pode provar que entende tudo do assunto!  
  
Tecle ENTER para continuar...█
```

Figura 5 – Tela de Introdução da Partida

Uma partida no Chute Certo é composta de 20 questões (desafios) que são apresentadas numa ordem aleatória para o usuário por meio do método `embaralharQuestoes()` que é executado na função `setup()`. A lógica de cada desafio é encapsulada na função `telaDesafio()`. Esta última é chamada dentro de um laço de repetição `for`, recebendo o índice correspondente à posição dos dados no array.

```
1      public static void telaDesafio(int indiceAtual) {
2          numeroDesafio = indiceAtual + 1;
3          perguntaDesafio = PERGUNTAS[indiceAtual];
4
5          // 1. Cria um vetor temporario para juntar a resposta
        certa e as 3 erradas
6          String[] opcoes = new String[4];
7          opcoes[0] = RESPOSTAS[indiceAtual];
8          opcoes[1] = ALTERNATIVAS[indiceAtual][0];
9          opcoes[2] = ALTERNATIVAS[indiceAtual][1];
10         opcoes[3] = ALTERNATIVAS[indiceAtual][2];
11
12         // 2. Randomiza a ordem das alternativas
13         Random random = new Random();
14         for (int i = 0; i < opcoes.length; i++) {
15             int pos = random.nextInt(opcoes.length-i) + i;
16             String temp = opcoes[i];
17             opcoes[i] = opcoes[pos];
18             opcoes[pos] = temp;
19         }
20
21         opcao1 = opcoes[0];
22         opcao2 = opcoes[1];
23         opcao3 = opcoes[2];
24         opcao4 = opcoes[3];
25
26         // 3. Descobre em qual numero a resposta certa caiu
27         for (int i = 0; i < 4; i++) {
28             if (opcoes[i].equals(RESPOSTAS[indiceAtual])) {
29                 resposta = i + 1;
30             }
31         }
32
33         desenharTela(TELA_DESAFIO);
```

Listing 2.5 – Primeira parte do código do desafio

No início de sua execução, a função realiza a atribuição de variáveis locais: define o número do desafio corrente (acrescentando 1 ao índice atual), captura o enunciado da questão, a resposta correta e as demais opções. Visando a imprevisibilidade do quiz, as alternativas são unificadas em um `array` e embaralhadas por meio de um objeto da classe `Random`. Na sequência, a função `desenharTela()` é acionada para exibir visualmente o desafio estruturado ao jogador, como ilustrado na Figura 6.

```
DESAFIO 2 de 20
Sua Pontuação: 0
-----
Qual operador Java é usado para comparar a igualdade de dois valores primitivos?
-----
A) equals()
B) ==
C) =
D) !=
Diga sua resposta: C
```

Figura 6 – Tela de um desafio do quiz

```
1      //recebe a entrada como letra
2      String escolhaLetra = lerOpcoes("Diga sua resposta", new
String[]{"A", "B", "C", "D"}, true, "\nDigite 'A', 'B', 'C' ou
'D' para escolher uma alternativa.").toUpperCase();
3
4      //mapeia a letra para o valor numerico correspondente
5      int escolhaNum = switch (escolhaLetra) {
6          case "A" -> 1;
7          case "B" -> 2;
8          case "C" -> 3;
9          default -> 4;
10     };
11
12     if(escolhaNum == resposta) {
13         pontuacao += 10;
14
15         System.out.println("\nG0000000000L!!!");
16         System.out.println("\nVocê chutou a bola e acertou no
gol.\nVocê ganhou " + 10 + " pontos!");
17         System.out.println("\nContinue assim, artilheiro!\n\n
");
18     } else {
19         System.out.println("\nErrou o chute!!!");
20         System.out.println("\nTudo bem, bola fora acontece. N
ão desista e continue chutando!!!\n");
21     }
```

```
22
23     pausar();
24 }
```

Listing 2.6 – Segunda parte do código do desafio

Conforme apresentado no Código 2.6, a segunda metade da função processa a resposta do usuário capturada em formato de texto (String), restringindo-se às alternativas "A", "B", "C" ou "D". Na sequência, uma expressão `switch` realiza o mapeamento dessas letras para seus respectivos valores numéricos (de 1 a 4).

Com base nesse valor, o sistema executa uma estrutura condicional para validar o palpite em relação à resposta correta. Caso o usuário acerte, a pontuação é incrementada em 10 pontos. Independentemente do resultado da validação, o sistema exibe um feedback visual temático e motivacional, invocando a função `pausar()` antes de avançar para o próximo desafio ou para a tela de encerramento da partida.

```
1     public static void telaFinalizar() {
2         //controle de exibição da pontuação na tela de score no
fim da partida
3         fimDaPartida = true;
4
5         desenharTela(TELA_FINALIZAR);
6
7         String nome = lerComPadrao("Digite seu nome para o Placar
: ", "^(?=.*[A-Za-z])[A-Za-z ]+$", "Por favor, utilize somente
letras!").trim();
8
9         //pega apenas as 3 primeiras letras
10        nome = nome.length() > 3 ? nome.substring(0, 3) : nome;
11
12        //salva a pontuação nos recordes se estiver no top 10
13        boolean scoreSalvo = salvarNoPlacar(nome, pontuacao);
14
15        desenharTela(TELA_SCORE);
16        desenharResultado(scoreSalvo);
17
18        fimDaPartida = false;
19    }
```

Listing 2.7 – Código da Tela de Finalização da Partida

Ao término do último desafio, o laço de repetição `for` é encerrado e a função `telaFinalizar()` é invocada (Código 2.7). Essa rotina solicita ao usuário a inserção de

um nome com o limite de até três caracteres, o qual é enviado como parâmetro para a função `salvarNoPlacar()`.

Esta última sub-rotina analisa a pontuação obtida pelo jogador em relação às marcas registradas no placar de líderes. Caso a pontuação atual supere qualquer uma das dez melhores posições, o novo registro é inserido na tabela de classificação e, consecutivamente, a menor pontuação é descartada para manter o limite fixo do ranking. Independentemente do resultado da validação no placar, uma mensagem motivacional de encerramento é exibida ao usuário.

Com isso, o ciclo principal do jogo é concluído. Após a exibição da mensagem final, a aplicação retorna ao menu principal (Tela de Título). Nessa interface, o usuário pode optar por reiniciar o ciclo de desafios ou fechar o programa.

2.2 Arquitetura e Modularização

2.2.1 Dados da Aplicação

O gerenciamento e a manipulação dos dados no *Chute Certo* baseiam-se em estruturas de dados estáticas e homogêneas. Para garantir a dinâmica do jogo e a consistência das rodadas, utilizou-se o conceito de **Vetores Paralelos** combinado a uma **Matriz Bidimensional** e a um algoritmo de permutação aleatória.

Como o sistema foi projetado para rodar em modo terminal sem persistência externa em bancos de dados, o armazenamento do banco de questões foi estruturado por meio de três coleções estáticas interligadas pelo índice do array:

- **PERGUNTAS:** Um vetor unidimensional de *Strings* que armazena os enunciados das questões.
- **RESPOSTAS:** Um vetor unidimensional de *Strings* contendo exclusivamente as respostas corretas de cada desafio.
- **ALTERNATIVAS:** Uma matriz bidimensional ($N \times 3$) de *Strings* que guarda as três opções incorretas de cada pergunta.

A integridade do relacionamento é mantida de forma posicional: a pergunta contida em `PERGUNTAS[i]` possui sua resposta correta em `RESPOSTAS[i]` e suas opções incorretas nas três colunas da linha `ALTERNATIVAS[i]`, conforme exemplificado na inicialização dessas estruturas:

```
1 public static final String[] PERGUNTAS = {
2     "Se o Brasil ganhou 5 Copas do Mundo e quer ganhar mais 2,
3     com quantas ficara? (Matematica)",
4     "Qual e o pais do futebol que fica na America do Sul e veste
5     verde e amarelo? (Geografia)"
6 };
7
8 public static final String[] RESPOSTAS = {
9     "7 Copas",
10    "Brasil"
11 };
12
13 public static final String[][] ALTERNATIVAS = {
14     {"6 Copas", "8 Copas", "5 Copas"},
15     {"Argentina", "Uruguai", "Colombia"}
16 };
```

Listing 2.8 – Modelagem de vetores paralelos e matriz de alternativas.

São utilizadas também outras variáveis e constantes globais para guardar dados de estado da aplicação, como a tela atual em que o usuário está no momento, sua pontuação, a pergunta e alternativas do desafio atual e os valores de nomes e pontos do Placar de Líderes:

```
1 /**
2     * Constante para guardar a quantidade de linhas para
3     * pular para fazer uma mudança de tela.
4     */
5 public static final int ALTURA_TELA = 30;
6
7 /**
8     * Constantes para mapeamento das telas do quiz.
9     */
10 public static final int TELA_TITULO = 1;
11 public static final int TELA_REGRAS = 2;
12 public static final int TELA_INTRODUCAO = 3;
13 public static final int TELA_DESAFIO = 4;
14 public static final int TELA_SCORE = 5;
15 public static final int TELA_FINALIZAR = 6;
16 public static final int TELA_SAIR = 0;
17
18 /**
19     * Variável que armazena o estado da tela atual do quiz.
```

```
20      */
21      public static int telaAtual = TELA_TITULO; // Ajustado para
22      iniciar na tela titulo logicamente
23
24      /**
25       * variáveis para armazenar informações do desafio atual
26       */
27      public static int numeroDesafio;
28      public static String perguntaDesafio;
29      public static String opcao1;
30      public static String opcao2;
31      public static String opcao3;
32      public static String opcao4;
33      public static int resposta;
34
35      /**
36       * controla o estado de exibição da pontuação no score no
37       final da partida
38       */
39      public static boolean fimDaPartida = false;
40
41      /**
42       * variáveis para armazenar os dados do score
43       */
44      public static String[] nomesTop10 = new String[10];
45      public static int[] pontosTop10 = new int[10];
46      public static int pontuacao;
```

Listing 2.9 – Outras constantes e variáveis globais

2.2.2 Separação de camada de renderização

Com o objetivo de garantir a manutenibilidade e a organização do código-fonte, a arquitetura da aplicação adota uma clara divisão entre o processamento lógico e a interface do usuário. Essa separação é evidenciada pela criação de métodos específicos de renderização, padronizados com o prefixo `desenhar`, como visto abaixo:

```
1      /**
2       * Desenha uma tela específica do quiz, limpando a tela antes
3       por padrão.
4       * @param codigoTela número que indica a tela a ser desenhada
5       .
6       */
```

```
5     public static void desenharTela(int codigoTela);
6
7     /**
8      * Desenha uma tela específica do quiz, utilizando uma ou
9      * mais funções de desenhar.
10     * @param codigoTela número que indica a tela a ser desenhada
11     */
12     public static void desenharTela(int codigoTela, boolean
13     limparTela);
14
15     /**
16     * Desenha na tela o título do jogo em ASCII Art
17     */
18     public static void desenharLogo();
19
20     /**
21     * Desenha o MENU INICIAL com as opções para o usuário.
22     */
23     public static void desenharMenuTitulo();
24
25     /**
26     * Desenha na tela o título do jogo em ASCII Art
27     */
28     public static void desenharDesafio();
29
30     /**
31     * Exibe as regras do jogo para o usuário
32     */
33     private static void desenharRegras();
34
35     /**
36     * Desenha a tela de introdução ao iniciar o jogo.
37     */
38     public static void desenharIntroducao();
39
40     /**
41     * Desenha a tela de finalização do jogo.
42     */
43     public static void desenharFinalizar();
44
45     /**
```

```
44      * Desenha a mensagem de sucesso ou falha ao tentar entrar no
      Placar de Líderes
45      * @param entrouNoPlacar informa se o jogador conseguiu ser
      salvo no placar.
46      */
47      public static void desenharResultado(boolean entrouNoPlacar);
48
49      /**
50      * Desenha a tela de score do jogo.
51      */
52      public static void desenharScore();
53
54      /**
55      * Desenha a tela de saída do jogo.
56      */
57      public static void desenharSair();
```

Listing 2.10 – Funções de Renderização

O escopo de execução dessas funções é restrito unicamente à apresentação do conteúdo textual na tela do usuário. Elas atuam de forma passiva: recebem os dados já processados pelas funções de controle de fluxo (como as variáveis de pontuação ou os enunciados das perguntas) e realizam as chamadas de impressão no console. Essa abordagem impede o acoplamento excessivo, o que significa que a lógica de pontuação ou o sorteio de questões não dependem de como a tela é desenhada. Como resultado, o sistema ganha em legibilidade, facilitando a depuração de erros e futuras expansões no Quiz Chute Certo.

2.2.3 Validação da entrada do usuário

Um dos critérios fundamentais na engenharia de software é a garantia da robustez e da resiliência da aplicação diante de entradas de dados imprevistas ou inválidas por parte do usuário. No contexto do Quiz Chute Certo, esse mecanismo de defesa foi centralizado em funções específicas de captura de dados, padronizadas com o prefixo `ler`. Essas rotinas interceptam a entrada do usuário inicialmente no formato de texto (`String`) e aplicam regras de validação distintas antes de liberar o dado para as demais camadas do sistema.

Como alicerce dessa estrutura, implementou-se a função base `lerString()`. A responsabilidade única deste método limita-se a realizar a leitura bruta do fluxo de entrada do console e retornar o valor capturado, sem aplicar qualquer tipo de restrição ou validação. Essa abstração permite que métodos de leitura mais complexos e especializados invoquem `lerString()` internamente, concentrando-se exclusivamente em suas respectivas regras de

validação e conversão de tipos (como checagem de limites numéricos ou formatação de caracteres).

```
1  /**
2   * Solicita a entrada para o usuário e devolve a string lida.
3   * @param textoParaEntrada texto para requisitar a entrada do
  usuário.
4   * @return retorna a string inserida pelo usuário.
5   */
6  public static String lerString(String textoParaEntrada) {
7      System.out.print(textoParaEntrada);
8      return ENTRADA.nextLine();
9  }
```

Listing 2.11 – Função lerString()

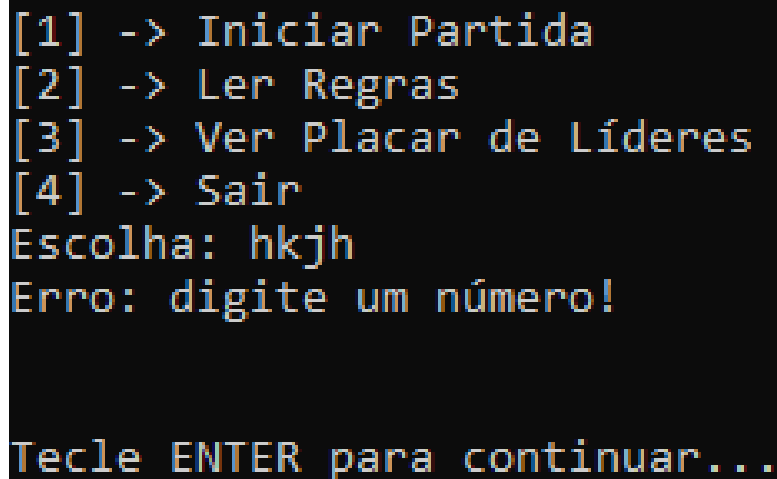
Por sua vez, as sobrecargas da função lerOpcoes() processam o retorno obtido por lerString() e validam se o valor inserido consta no conjunto de opções permitidas, utilizando o método utilitário contemValor(). Caso o usuário forneça um dado inválido — como uma opção fora do escopo ou uma entrada não numérica onde se espera um inteiro —, o mecanismo de tratamento de erros handleError() é disparado. No caso da sobrecarga que manipula inteiros, o fluxo realiza adicionalmente o parsing do texto para o tipo primitivo int utilizando Integer.parseInt(). Esta sub-rotina exibe uma mensagem de alerta ao usuário, aciona a renderização da telaAtual e solicita novamente a entrada. Esse fluxo de controle permanece em um laço de repetição iterativo (ciclo de validação) até que uma entrada plenamente válida seja fornecida, assegurando a integridade dos dados na aplicação.

```
1  /**
2   * Solicita entrada de um inteiro para o usuário que esteja
  dentro do array de opções válidas
3   * e reexibe a interface, solicitando novamente a entrada,
  caso o usuário insira valor inválido.
4   * @param textoParaEntrada texto exibido ao solicitar a
  entrada.
5   * @param valoresValidos array de inteiros com as opções
  aceitas.
6   * @return o valor inteiro válido inserido pelo usuário.
7   */
8  public static int lerOpcoes(String textoParaEntrada, int[]
  valoresValidos, String mensagemErro) {
9      textoParaEntrada = adicionarDoisPontos(textoParaEntrada);
10
11      while(true) {
```

```
12         boolean erroFormato = false;
13         String entradaUsuario = lerString(textoParaEntrada);
14
15         try {
16             int num = Integer.parseInt(entradaUsuario.trim())
17         ;
18             if(contemValor(num, valoresValidos)) {
19                 return num;
20             }
21         } catch (NumberFormatException e) {
22             erroFormato = true;
23         }
24         handleErro(erroFormato ? "Erro: digite um número!" :
25         mensagemErro);
26     }
27 }
```

Listing 2.12 – Função lerOpcoes()

Conforme demonstrado na /Ref 7 vemos na pratica o funcionamento de acordo com uma entrada invalida de dados.



```
[1] -> Iniciar Partida
[2] -> Ler Regras
[3] -> Ver Placar de Líderes
[4] -> Sair
Escolha: hkjh
Erro: digite um número!

Tecle ENTER para continuar...
```

Figura 7 – Entrada de dados incorretos

As outras funções de leitura são sobrecarga da função lerComPadrao(), que é muito semelhante à função anterior, ela não recebe um array de valores válidos mas uma regex que define o padrão de texto que será aceito. Dentro da função é utilizado String.matches para

validar o formato. Em caso de entrada inválida, o mesmo fluxo é realizado, redesenhando a tela e solicitando nova entrada.

```
1 /**
2  * Solicita entrada de uma string para o usuário que esteja
3  * dentro do padrão especificada (regex).
4  * @param textoParaEntrada texto exibido ao solicitar a
5  * entrada.
6  * @param pattern regex especificando o padrão de entrada
7  * esperado.
8  * @return a string válida inserida pelo usuário.
9  */
10 public static String lerComPadrao(String textoParaEntrada,
11 String pattern, String mensagemErro) {
12     textoParaEntrada = adicionarDoisPontos(textoParaEntrada);
13
14     while(true) {
15         String entradaUsuario = lerString(textoParaEntrada);
16         if(entradaUsuario.matches(pattern)) {
17             return entradaUsuario;
18         }
19         handleErro(mensagemErro);
20     }
21 }
```

Listing 2.13 – Função lerComPadrao()

2.2.4 Algoritmo de Fisher-Yates

Para evitar que o jogo se torne previsível e garantir o seu valor pedagógico em múltiplas utilizações pelos alunos, implementou-se o método `embaralharQuestoes()`. Esse método utiliza uma variação do algoritmo clássico de **Fisher-Yates**:

```
1 public static void embaralharQuestoes(String[] questoes, String
2   [][] alternativas, String[] respostas) {
3     Random random = new Random();
4
5     for(int i = 0; i < questoes.length - 1; i++) {
6         int j = random.nextInt(questoes.length - i) + i;
7
8         // Swap no vetor de perguntas
9         String tempQuestoes = questoes[i];
10        questoes[i] = questoes[j];
11        questoes[j] = tempQuestoes;
```

```
11
12     // Swap no vetor de respostas
13     String tempRespostas = respostas[i];
14     respostas[i] = respostas[j];
15     respostas[j] = tempRespostas;
16
17     // Swap na matriz de alternativas (troca a linha inteira)
18     String[] tempAlternativa = alternativas[i];
19     alternativas[i] = alternativas[j];
20     alternativas[j] = tempAlternativa;
21 }
22 }
```

Listing 2.14 – Algoritmo de permutação simultânea baseado em Fisher-Yates.

O algoritmo opera realizando uma varredura do fim para o início dos arrays, escolhendo um índice aleatório gerado pela classe `java.util.Random` e realizando a troca (*swap*) de elementos. O diferencial técnico desta implementação reside no fato de realizar o *swap* simultâneo nas três estruturas paralelas. Isso garante que, mesmo mudando a pergunta de posição no vetor, ela nunca perca o vínculo com sua resposta certa e suas respectivas alternativas distratoras.

Como visto na seção anterior, além de embaralhar a ordem em que as perguntas aparecem, o sistema executa um segundo algoritmo de randomização local dentro do método `telaDesafio()` (código 2.5). Ele unifica a resposta correta e as três incorretas em um vetor temporário de 4 posições e rearranja a ordem de exibição (A, B, C ou D).

2.2.5 Função adicionarDoisPontos()

Este método atua na sanitização e padronização estética das interfaces de texto da aplicação. Ele utiliza a função nativa `.trim()` para eliminar espaçamentos em branco sobressalentes nas extremidades da String recebida e aplica uma expressão regular para verificar se o texto já termina com o caractere de dois-pontos; caso não termine, concatena os caracteres ao fim da mensagem, mitigando erros de digitação por parte dos desenvolvedores e garantindo consistência visual em todos os prompts do console.

```
1  /**
2   * Verifica se o texto possui ": " no final, adicionando-o se
   não tiver.
3   * @param texto texto que vai ser verificado e formatado.
4   * @return o texto formatado com ": " no final.
5   */
6  public static String adicionarDoisPontos(String texto) {
```

```
7      // retira os espaços do início e fim
8      texto = texto.trim();
9
10     // verifica se não tem ':' no final
11     if(!texto.matches("^.*:$")){
12         texto += ": ";
13     } else {
14         texto += " ";
15     }
16
17     return texto;
18 }
```

Listing 2.15 – Método utilitário para formatação e sanitização de prompts de texto

2.2.6 Função limparTela()

Responsável por gerenciar a experiência do usuário (UX) eliminando a poluição visual do terminal, este método emula a limpeza da tela de forma totalmente portátil e agnóstica. Em vez de depender de comandos nativos do sistema operacional subjacente (como `cls` ou `clear`), a função utiliza um laço de repetição `for` atrelado a uma constante de altura para injetar sucessivas linhas em branco (`System.out.println()`) no console, deslocando o conteúdo anterior para fora do limite visível do jogador e preparando a interface para uma nova renderização.

```
1      public static void limparTela() {
2          for(int i = 0; i < ALTURA_TELA; i++) {
3              System.out.println();
4          }
5      }
```

Listing 2.16 – Método portátil para emulação de limpeza de tela via console

2.2.7 Função pausar()

Desenvolvido para controlar o ritmo de transição do jogo (pacing), este método interrompe temporariamente o fluxo de execução do programa após a exibição de feedbacks importantes (como o resultado de uma questão ou pontuação). Evidenciando o reaproveitamento de código, a função invoca internamente o método base `lerString()` passando a mensagem instrucional "Tecle ENTER para continuar...", o que retém o sistema em estado de espera até que uma ação explícita de quebra de linha `()` seja fornecida pelo usuário.

```
1  /**
2   * Espera o usuário digitar "Enter" para continuar o jogo.
3   */
4  public static void pausar() {
5      lerString("Tecle ENTER para continuar...");
6  }
```

Listing 2.17 – Método de controle de fluxo para interrupção temporária da execução

2.2.8 Função `contemValor()`

Inteiros: Esta assinatura implementa um algoritmo clássico de busca linear (ou sequencial) focado em tipos primitivos inteiros (`int`). Utilizando uma estrutura de repetição aprimorada (`foreach`), o método varre um vetor de inteiros fornecido como parâmetro e compara cada elemento com o valor buscado; caso encontre uma correspondência exata, interrompe a execução retornando o booleano `true`, finalizando com o retorno `false` apenas se percorrer todo o array sem sucesso.

Strings: Constituindo uma sobrecarga do método anterior (polimorfismo estático), esta assinatura estende a busca sequencial para vetores de texto (`String[]`) e adiciona flexibilidade através de um parâmetro booleano de controle de caixa. No interior do laço de repetição, a função avalia as strings por meio de um operador ternário: caso a flag indique para ignorar a caixa, a validação é delegada ao método `.equalsIgnoreCase()`; caso contrário, aplica-se a comparação estrita com o `.equals()`, permitindo validar de forma genérica tanto comandos rígidos quanto respostas flexíveis do usuário.

```
1  /**
2   * Busca um valor dentro de um array de inteiro e retorna
3   * true se for encontrado, senão retorna false.
4   * @param busca valor para ser procurado no array.
5   * @param fonte array no qual a procura será feita.
6   * @return true ou false se o valor foi encontrado.
7   */
8  public static boolean contemValor(int busca, int[] fonte) {
9      for(int valor : fonte) {
10         if(valor == busca)
11             return true;
12     }
13     return false;
14 }
15
16 /**
17  * Busca um valor dentro de um array de String e retorna true
18  * se for encontrado, senão retorna false.
19  * @param busca valor para ser procurado no array.
20  * @param fonte array no qual a procura será feita.
21  * @param ignorarCaixa indica se deve diferenciar maiúsculas
22  * e minúsculas na busca ou não.
23  * @return true ou false se o valor foi encontrado.
24  */
25 public static boolean contemValor(String busca, String[]
26 fonte, boolean ignorarCaixa) {
27     for(String valor : fonte) {
28         if(ignorarCaixa ? valor.equalsIgnoreCase(busca) :
29         valor.equals(busca)) {
30             return true;
31         }
32     }
33     return false;
34 }
```

Listing 2.18 – Sobrecargas do método utilitário de busca sequencial em arrays

2.3 Código Fonte

Com o objetivo de garantir a transparência acadêmica, o versionamento do projeto CHUTE CERTO o código-fonte integral da aplicação foi disponibilizado em um repositório público na plataforma GitHub, acessível pelo endereço abaixo:

Repositório do Projeto no GitHub:

O código-fonte completo, o arquivo executável compilado e o histórico de versionamento deste sistema encontram-se disponíveis publicamente para fins de auditoria acadêmica no repositório oficial da plataforma GitHub através do seguinte endereço eletrônico:

<<https://github.com/Allan-Jackson/chute-certo/blob/main/src/br/com/senac/begods/chutecerto/Main.java>>

3 Conclusão

O desenvolvimento do projeto permitiu aplicar diversos conceitos fundamentais da programação em Java, incluindo modularização, manipulação de vetores, matrizes, métodos e validação de dados.

Além do aprendizado técnico, o projeto também proporcionou experiência prática na organização de código e desenvolvimento de aplicações interativas em terminal.

Referências